

Homework 4: Context-Free Languages

Will Griffin

February 19, 2026

1. Arithmetic Expressions

Consider the grammar G_4 for arithmetic expressions, with start symbol E :

$$\begin{aligned}E &\longrightarrow E + T \mid T \\T &\longrightarrow T \star F \mid F \\F &\longrightarrow (E) \mid \mathbf{a} \mid \mathbf{b} \mid \mathbf{c}\end{aligned}$$

- (a) Give derivations for the following strings. You may write them as a sequence of rewrites or as a tree.

- (i) $\mathbf{a} + \mathbf{b} + \mathbf{c}$

$$\begin{aligned}E &\Longrightarrow E + T \\&\Longrightarrow E + T + T \\&\Longrightarrow T + T + T \\&\Longrightarrow F + T + T \\&\Longrightarrow F + F + T \\&\Longrightarrow F + F + F \\&\Longrightarrow \mathbf{a} + F + F \\&\Longrightarrow \mathbf{a} + \mathbf{b} + F \\&\Longrightarrow \mathbf{a} + \mathbf{b} + \mathbf{c}\end{aligned}$$

- (ii) $\mathbf{a} \star \mathbf{b} + \mathbf{c}$

$$\begin{aligned}E &\Longrightarrow E + T \\&\Longrightarrow T + T \\&\Longrightarrow T \star F + T \\&\Longrightarrow F \star F + T \\&\Longrightarrow F \star F + F \\&\Longrightarrow \mathbf{a} \star F + F \\&\Longrightarrow \mathbf{a} \star \mathbf{b} + F \\&\Longrightarrow \mathbf{a} \star \mathbf{b} + \mathbf{c}\end{aligned}$$

- (iii) $\mathbf{a} \star (\mathbf{b} + \mathbf{c})$

$$\begin{aligned}
E &\Rightarrow T \\
&\Rightarrow T \star F \\
&\Rightarrow T \star (E) \\
&\Rightarrow T \star (E + T) \\
&\Rightarrow T \star (T + T) \\
&\Rightarrow F \star (T + T) \\
&\Rightarrow F \star (F + T) \\
&\Rightarrow F \star (F + F) \\
&\Rightarrow a \star (F + F) \\
&\Rightarrow a \star (b + F) \\
&\Rightarrow a \star (b + c)
\end{aligned}$$

(b) Modify G_4 to allow an exponentiation operator $\uparrow$.

- (a) It should have *higher precedence* than multiplication. For $a \star b \uparrow c$, there should be a nonterminal that rewrites to $b \uparrow c$, and not a nonterminal that rewrites to $a \star b$.
- (b) It should be *right-associative*, that is, in the derivation of $a \uparrow b \uparrow c$, there should be a nonterminal that rewrites to $b \uparrow c$ and not one that rewrites to $a \uparrow b$.

A grammar to allow exponentiation, G_4 , can be described as such:

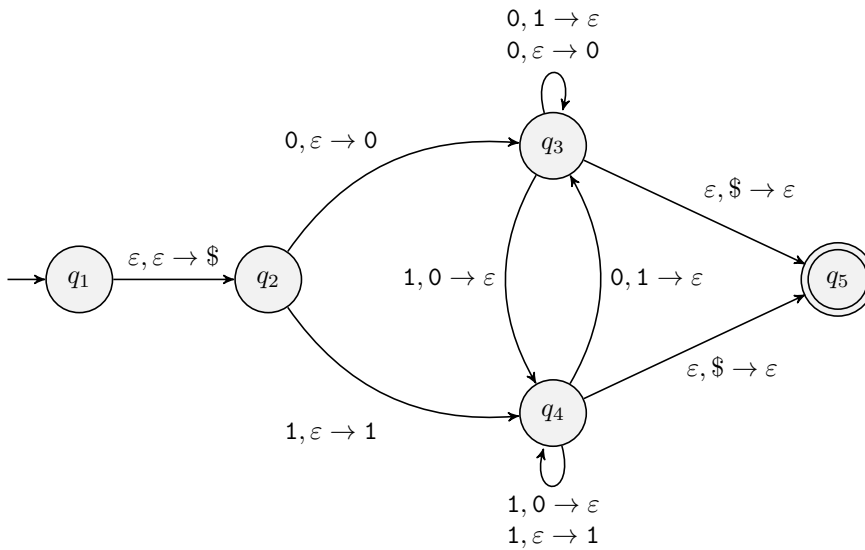
$$\begin{aligned}
E &\longrightarrow E + T \mid T \\
T &\longrightarrow T \star B \mid B \\
B &\longrightarrow F \uparrow B \mid F \\
F &\longrightarrow (E) \mid a \mid b \mid c
\end{aligned}$$

2. String with equal 0s and 1s

Prove the following language to be context-free by writing either a PDA *or* a CFG for it:

$$C = \{w \in \{0, 1\}^* \mid w \text{ has an equal number of 0s and 1s}\}$$

The PDA M can be used to describe the language C :



This automata describes the language C . The PDA's stack is used to keep track of the imbalance of 0s and 1s, and when a 0 or a 1 is read, if there is an opposing 1 or 0 not on the stack, it itself is added to the stack. Otherwise, the opposing symbol is removed from the stack. Since you can only reach the accept state by popping a $\$$ from the stack, there must be an equal number of 0s and 1s in the string.

3. Language Complement

Prove the following language to be context-free by writing either a PDA *or* a CFG for it:

$$L_3 = \overline{\{0^n 1^n \mid n \geq 0\}}$$

For example, $000111 \notin L_3$. Hint: First prove that this is equal to $\{0^m 1^n \mid m \neq n\} \cup \overline{0^* 1^*}$.

L_3 covers two cases of $\{0, 1\}^*$:

1. The number of 0s cannot be equal to the number of 1s. Therefore, any string of the form $0^n 1^n$ where $n \geq 0$ cannot be in the set.
2. There must be at least one 0 following a 1. Therefore, any string of the form $0^* 1^*$ cannot be in the set.

Using these characteristics, it is clear that $L_3 = \{0^m 1^n \mid m \neq n\} \cup \overline{0^* 1^*}$.

Seeing this, we can construct the grammar G_3 to generate strings that belong to L_3 with start state S :

$$\begin{aligned} S &\longrightarrow Z \mid O \mid P \\ Z &\longrightarrow 0Z \mid 0A \\ A &\longrightarrow 0A1 \mid \varepsilon \\ O &\longrightarrow O1 \mid B1 \\ B &\longrightarrow 0B1 \mid \varepsilon \\ P &\longrightarrow C10C \\ C &\longrightarrow 0C \mid C1 \mid \varepsilon \end{aligned}$$

The Z rule is to ensure there are always more 0s than 1s. By forcing the extra 0s, whatever result is generated by the Z rule will be in L_3 .

The O rule is to ensure there are always more 1s than 0s. By forcing the extra 1s, whatever result is generated by the O rule will be in L_3 .

The P rule is to ensure a 0 will always follow at least one 1. Covering this case, the generated string will be a member of $\overline{0^* 1^*}$, and therefore a member of L_3 .